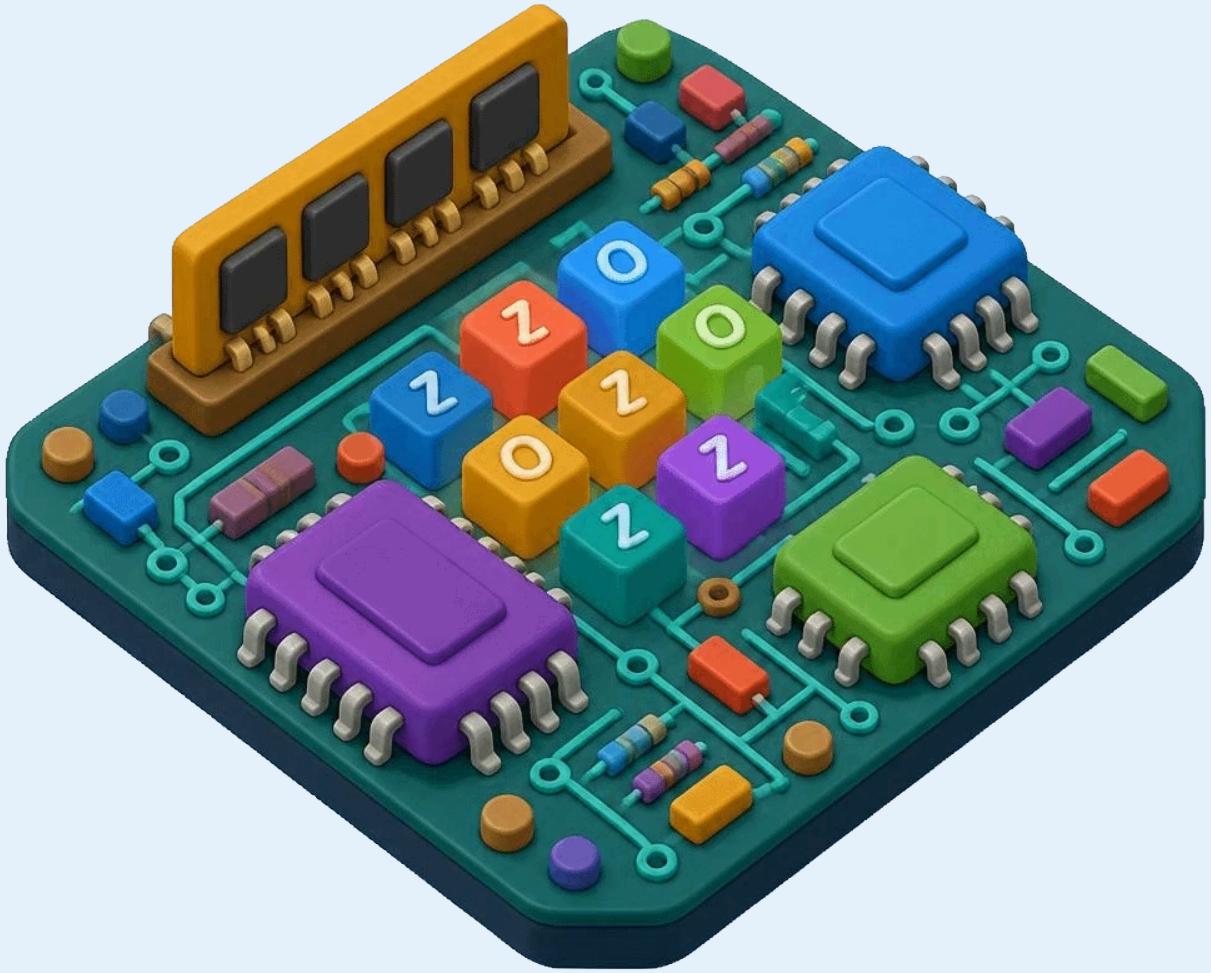
Представление строк в памяти компьютера, ASCII и Unicode



Made with GAMMA

ОСНОВЫ

Компьютер хранит всё в нулях и единицах

Бит

Минимальная единица информации: 0 или 1.

Байт

Группа из 8 бит, базовая ячейка памяти.

Пример байта

01000001 — число или символ в зависимости от кодировки.

Вопрос студентам: как именно из нулей и единиц получается обычный понятный человеку текст?



Made with GAMMA

ЦЕПОЧКА

Символ превращается в число, а число в биты

Преобразование символа в физическое состояние памяти проходит три строгих этапа на примере буквы A:

1

Символ → Код

'A' → 65

Символу таблицы сопоставляется порядковый номер по стандарту кодирования.

2

Число → Биты

65 → 01000001_2

Десятичный код переводится в двоичную позиционную систему счисления.

3

Запись в память

1 байт в RAM

8 бит физически сохраняются в ячейке оперативной памяти или накопителя.

Made with GAMMA

СТАНДАРТЫ

Кодирование символов: ASCII, Unicode, UTF-8

- **ASCII** — первоначально создан для текстов на английском языке.
- **Unicode** — единый каталог для практически всех письменностей мира.
- **UTF-8** — компактный способ записи кодов Unicode в байтах.

Один и тот же числовой код в совместимых стандартах обозначает один символ, но форма хранения в байтах может отличаться.

Символ	Код	Совместимость
A	65	ASCII, Unicode, UTF-8 (1 байт)
B	66	ASCII, Unicode, UTF-8 (1 байт)
C	67	ASCII, Unicode, UTF-8 (1 байт)
a	97	ASCII, Unicode, UTF-8 (1 байт)
b	98	ASCII, Unicode, UTF-8 (1 байт)

Made with GAMMA

ИСТОРИЯ И СТРУКТУРА

ASCII: 128 кодов для английского текста

Американский стандарт 1960-х годов: использует 7 бит на символ, всего 128 допустимых значений (коды 0–127).

0–31

Управляющие коды

Служебные сигналы телетайпов: перевод строки (LF), возврат каретки (CR), табуляция, звуковой сигнал.

32–126

Печатные символы

Пробел (код 32), цифры 0–9, буквы английского алфавита A–Z и a–z, базовые знаки пунктуации.

127

Символ DEL

Удаление символа (Delete), исторически пробивавший все 7 отверстий на перфоленте.

Made with GAMMA

СТРУКТУРА ТАБЛИЦЫ

Закономерности в кодах ASCII

- **Цифры 0–9:** коды от 48 до 57.
- **Заглавные латинские:** коды от 65 до 90.
- **Строчные латинские:** коды от 97 до 122.
- **Сдвиг регистра:** разница между строчной и заглавной буквой всегда ровно **32** (меняется 1 бит).

Задание: чему равен код буквы D, если код A равен 65?

			битный код
Пробел	32	0x20	010 0000
0	48	0x30	011 0000
1	49	0x31	011 0001
A	65	0x41	100 0001
B	66	0x42	100 0010
C	67	0x43	100 0011
a	97	0x61	110 0001
b	98	0x62	110

Made with GAMMA

ОГРАНИЧЕНИЯ

ASCII не хватает для других языков

128 ячеек вмещали только базовый английский алфавит. Мировая письменность — за рамками стандарта:

Нет кириллицы: А, Б, В, Я и весь славянский пласт отсутствуют.

Нет умлаутов и диакритики: немецкие ä, ö, ü и французские акценты не поместились.

Нет восточной письменности: китайские иероглифы 你好 требуют тысяч кодов.

Нет современных символов: валюты, знаки и эмодзи (😀, 🌍) появились позже.

Хаос несовместимых локальных кодировок (Windows-1251, KOI8-R) потребовал единого стандарта — **Unicode**.



Made with GAMMA

ГЛОБАЛЬНЫЙ СТАНДАРТ

Unicode: уникальный код для каждого символа мира

- Разработка с конца 1980-х, первая версия — **1991 год**.
- Включает латиницу, кириллицу, арабское письмо, иероглифы, математические знаки и эмодзи.
- Обозначение U+ означает, что дальше записан **шестнадцатеричный код** символа.

A → U+0041

Латиница

Б → U+0411

Я → U+042F

€ → U+20AC

Символ евро

😀 → U+1F600

Эмодзи смайлик



Made with GAMMA

ПРЕЕМСТВЕННОСТЬ

ASCII и Unicode: первые 128 кодов совпадают

Создатели Unicode намеренно сохранили полную преемственность: исторический текст в ASCII является валидным текстом Unicode.

Критерий	ASCII	Unicode
Количество кодов	128 (0–127)	Более 1 100 000 (17 плоскостей)
Поддержка языков	Только базовый английский	Английский, русский, китайский, арабский и сотни других
Эмодзи и знаки	Отсутствуют	Тысячи пиктограмм, эмодзи и спецсимволов
Назначение	Английский телетайпный текст	Универсальный международный обмен текстом
Пример совместимости	A = 65; B = 66	A = U+0041 (65); B = U+0042 (66)

Made with GAMMA

ПРАКТИКА ХРАНЕНИЯ

UTF-8 превращает коды Unicode в байты

UTF-8 — формат переменной длины: тратит от 1 до 4 байтов в зависимости от порядкового номера символа.

Символ	Код Unicode	Размер в UTF-8	Особенность кодирования
A	U+0041	**1 байт**	Полное совпадение со старым ASCII (01000001)
Б	U+0411	**2 байта**	Кириллица, греческий, арабский, иврит
€	U+20AC	**3 байта**	Специальные знаки, китайские и японские иероглифы
😀	U+1F600	**4 байта**	Эмодзи, редкие исторические письменности

Unicode: определяет, какие символы и числовые коды существуют в мире. ASCII — первые 128 символов Unicode.

UTF-8: определяет, как эти коды оптимально упаковываются в байты памяти.

Made with GAMMA

Как Python работает со строками?

В Python 3 строки работают с Unicode. Это позволяет использовать в одной строке символы разных языков.

```
text = "Hello Привет Мир"
print(text)
```

Результат: Hello Привет Мир

В Python строка представляет собой последовательность Unicode-кодовых точек.

```
text = "Python"
print(len(text))  # 6
text = "Привет"
print(len(text))  # 6
```

Важно: len() считает количество Unicode-кодовых точек, а не количество байтов UTF-8.

Например:

```
text = "Б"
print(len(text))  # 1
print(len(text.encode()))  # 2
```

Один символ, но два байта в UTF-8!

Made with GAMMA

Функция ord()

Назначение: возвращает числовой код Unicode одного символа. Синтаксис: ord(character)

Примеры (каждый print на отдельной строке):

```
print(ord("A"))  # 65
print(ord("B"))  # 66
print(ord("a"))  # 97
print(ord("Б"))  # 1041
print(ord("Я"))  # 1071
```

Обратите внимание: функция принимает строку, содержащую ровно один Unicode-кодовый символ.

Задание №1. Напишите программу, которая:

1. Запрашивает у пользователя один символ.
2. Получает его числовой код с помощью ord().
3. Выводит полученное значение.

Пример:

```
Введите символ: Б
Код символа: 1041
```

Made with GAMMA

Функция chr()

Назначение: преобразует числовой код Unicode в соответствующий символ. Синтаксис: chr(number)

```
print(chr(65))  # A
print(chr(66))  # B
print(chr(97))  # a
print(chr(1041))  # Б
```

Функция chr() выполняет обратное действие по отношению к ord().

- Функция ord(): A → 65, символ преобразуется в число.
- Функция chr(): 65 → A, число преобразуется в символ.

Задание №2. Попросите пользователя ввести целое число. Выведите символ, которому соответствует данный код Unicode.

```
Введите код: 1041
Символ: Б
```

Made with GAMMA

Практика: исследуем символы строки

Теперь объединим знания о строках, индексах и функции ord(). Задача: получить числовой код каждого символа введённого слова.

Пример:

```
text = "ABC"
print(ord(text[0]))
print(ord(text[1]))
print(ord(text[2]))
```

Результат: 65, 66, 67

Задание №3. Пользователь вводит слово. Напишите программу, которая:

1. Определяет длину строки с помощью len().
2. Получает каждый символ по индексу.
3. Выводит символ и его числовой код.

Пример результата:

Слово: CAT
C — 67
A — 65
T — 84

Made with GAMMA

Практика: создаём последовательность символов

С помощью chr() можно создавать символы по их числовым кодам. Например, выведем все заглавные латинские буквы:

```
for code in range(65, 91):
    print(chr(code), end=" ")
```

Результат: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Задания №4–6:

1. Выведите все строчные латинские буквы, используя коды от 97 до 122.
2. Выведите все цифры от 0 до 9, используя коды от 48 до 57.
3. Попросите пользователя ввести число и выведите 10 символов, начиная с указанного кода.

Made with GAMMA